

Lab 2, CSE121, Spring '26

GDB and Humidity/Temp Sensing

Instructor: Marcelo Siero

Author: Tharuka Kodituwakku

Part 2.1

I was able to create a binary from the .elf file in Google Drive. I initially had an error while trying to flash the file because I forgot to source esp-idf (`$ . ~/esp/export.sh`). `write-flash` was not a recognized command, I adjusted that to `write_flash`. After sourcing, I ran `$ idf.py set-target esp32c3`, then ran OpenOCD. I opened a new terminal tab to run gdb with the gdbinit file. I removed `mon reset halt`, which improved GDB's performance. I was also given a warning that `flushregs` is a deprecated alias, so I replaced that line with `maintenance flush register-cache`. After running that command, GDB returned:

Breakpoint 1 at 0x4200bbb8

In another new terminal tab, I ran `$ screen /ttyACM0 115200` to monitor the ESP's serial port. The message I repeatedly got back is:

```
result of compute is 35
```

I Ctrl+C'd the gdb tab and ran `(gdb) b compute`, which returned

Breakpoint 2 at 0x4200bba6

I ran `(gdb) info address compute`, which returned:

Symbol "compute" is at 0x4200bb7e in a file compiled without debugging.

I then ran `(gdb) info registers a0 a1 a2` and got back:

```
a0 0x23 35
```

```
a1 0x1 1
```

a2 0x1 1

To that end...

Compute 1st argument is 35 and is passed via register a0

Compute 2nd argument is 1 and is passed via register a1

Return value is 35 and is returned via register a0

The entry point of the “compute” function is at address

0x4200bb7e

Part 2.2

Before I could even begin to work on this part, I experienced a significant problem with my ESP. It stopped appearing in lsusb. What I had to do to reset it was reboot the Pi several times, and press Boot+Reset on the ESP; as well as letting it sit unplugged from the Pi while it “cooled down” (i.e. to let the capacitors discharge fully). This problem proceeded to reappear several times while working on this assignment. I finally got this to reliably resolve by frequently unplugging the ESP. Running `$ idf.py build` was a good time to leave the ESP to rest disconnected.

To write the actual program, I used the i2c_basic example project in the esp-idf repository. The example is designed to demonstrate the IMU. The IMU is connected to sensor address 0x68. It uses another address from the same GPIO pin to manage its power and identify itself (“Who_Am_I” functionality). That is not exactly how the SHTC3 (temp/humidity sensor) works. It uses the same GPIO pins for I2C, and address 0x70.

I had experienced significant difficulties getting the serial output to be anything other than continuous 0's. Proper timing and `vTaskDelay` intervals made the signals spaced out enough to not cause any errors. However, the real data started constantly being F1F1F1F1F1F1. The first byte starting with F indicated that the sensor readings were not being sent through, but instead a source of constant high signals. What turned out to be the issue is that I was using the wrong GPIO pins for my board. I have a Rust 2.0 Board, which uses pins 7 and 8 for I2C SCL/SDA. I was incorrectly using pins 8 and 10, which are proper for the Rust 1.0 board. I also forced GPIO 7 to be constantly high, thinking it's a power source toggle for the temperature sensor. That explains the F1F1 strings; GPIO 7 was constantly reading 1111. Printing the raw output to serial was a feature instrumental for debugging help.

AI Usage

<https://gemini.google.com/share/85397a30441b>

Other Sources

https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c3/esp32-c3-devkit-rust-2/user_guide.html#start-application-development

https://github.com/espressif/esp-idf/tree/master/examples/peripherals/i2c/i2c_basic